UNIVERSIDAD NACIONAL JOSÉ MARÍA ARGUEDAS FACULTAD DE INGENIERIA DE SISTEMAS

FACULTAD DE INGENIERÍA · ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS

ASIGNATURA: COMPUTACIÓN GRÁFICA (IIAD65) · SEMESTRE 2026-II

SOLAR 2D ANIMADO: VISUALIZACIÓN DE TRANSFORMACIONES MATRICIALES PARA ESTUDIANTES DE PRIMARIA

PRESENTADO POR:

Flores Arce, James Bryan

Atoccsa Muriel, Piero Jean Pier

Huaraca Romero, Isaac Taffarel

DOCENTE:

Mtro. Jhamer Anderson Rojas Silva

Andahuaylas - Apurímac - Perú 2026

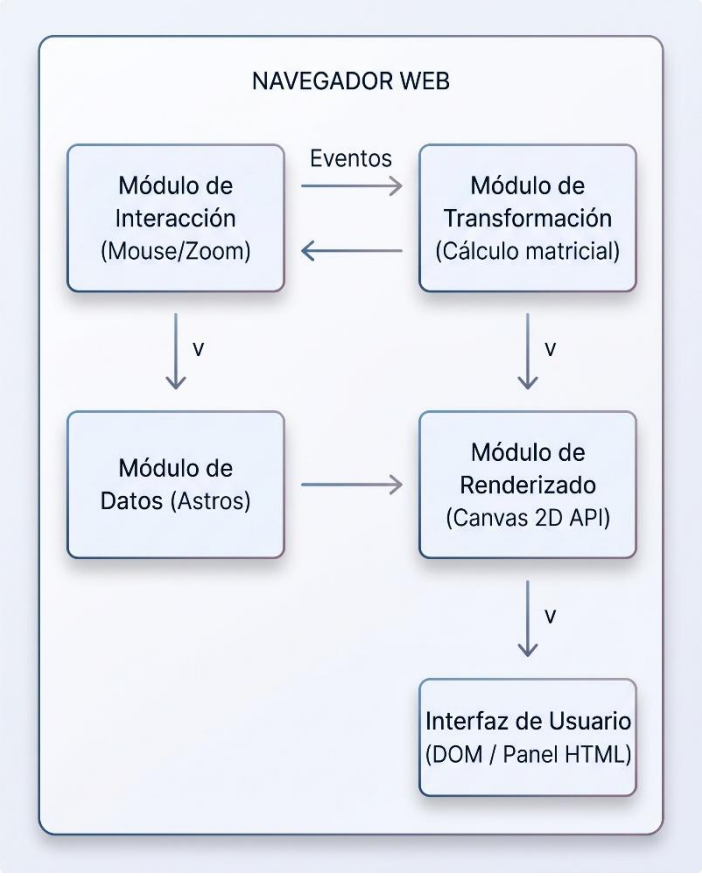
1. TECNOLOGÍAS Y VERSIONES

El proyecto fue desarrollado utilizando tecnologías web nativas (Vanilla), lo que garantiza que no requiera instalaciones de dependencias externas, facilitando su ejecución en los laboratorios de la universidad o en cualquier computadora con un navegador.

- **Lenguaje de programación:** JavaScript (ECMAScript 6+ / ES6).
- **Estructura y Diseño:** HTML5 y CSS3 (Flexbox, Grid, animaciones y transiciones).
- **Motor Gráfico:** HTML5 Canvas API (Contexto 2D). *No se utilizaron librerías como p5.js, Three.js ni WebGL, cumpliendo la regla de programar los algoritmos propios.*
- **Herramientas de desarrollo:** Visual Studio Code, Google Chrome (DevTools para pruebas).
- **Control de versiones:** Git y GitHub.

2. ARQUITECTURA DEL SISTEMA

El programa se ejecuta íntegramente en el lado del cliente (Frontend). La arquitectura es monolítica en su archivo principal, pero está lógicamente dividida en módulos funcionales que interactúan entre sí dentro del bucle de animación.



Conectividad de los módulos:

1. **Datos:** Mantiene el estado (ángulos, velocidades, padres).
2. **Transformación:** Lee los datos y calcula las nuevas coordenadas (x, y) de cada astro.
3. **Renderizado:** Toma las coordenadas calculadas y dibuja los píxeles en el Canvas.
4. **Interacción:** Escucha al usuario (clics, rueda, teclas) y modifica el estado (zoom, pan, selección).

3. PIPELINE GRÁFICO

El siguiente diagrama de flujo muestra el orden de ejecución en cada fotograma (frame) del programa, ejecutado a través de requestAnimationFrame:



4. DESCRIPCIÓN DE MÓDULOS Y ALGORITMOS

4.1 Módulo de Datos (celestialBodies)

- **Qué hace:** Almacena un arreglo de objetos JSON con las propiedades de más de 20 cuerpos celestes. Cada objeto contiene datos físicos reales (realRadius, temp, facts) y datos para la simulación (radius, distance, baseSpeed, parentId).
- **Algoritmo/Justificación:** Se utiliza una estructura de árbol (grafo dirigido) mediante la propiedad parentId. Esto permite que el algoritmo de posicionamiento sepa de quién depende cada astro (ej. La Luna depende de la Tierra).

4.2 Módulo de Transformación (Cálculo Matricial Anidado)

- **Qué hace:** Calcula las coordenadas (x, y) de cada cuerpo en la pantalla en tiempo real.
- **Fórmulas y Lógica:** Se aplica la composición de matrices en coordenadas homogéneas.

4.3 Módulo de Renderizado Vectorial

- **Qué hace:** Dibuja los planetas, el Sol y las lunas en el Canvas.
- **Algoritmos:**
    - **Rasterización de círculos:** Implementada nativamente por ctx.arc().
    - **Anillos de Saturno (Elipses):** Se usa ctx.ellipse() y se divide en dos mitades (trasera y delantera) para dar el efecto de que el anillo pasa por detrás del planeta.
    - **Órbitas:** Se utiliza ctx.setLineDash([4, 6]) para crear el efecto de línea punteada didáctica.

4.4 Módulo de Interacción y Detección de Colisiones

- **Qué hace:** Gestiona el zoom, el paneo (arrastre) y la selección de astros.
- **Algoritmo de Colisión (Hit Testing):** Cuando el usuario hace clic, se calcula la distancia euclidiana entre el clic y el centro del astro usando el Teorema de Pitágoras (Math.hypot). Si la distancia es menor al radio del astro (escalado por el zoom), se selecciona el objeto.

5. FRAGMENTOS DE CÓDIGO CLAVE

5.1 Transformación Matricial Anidada (El núcleo matemático)

Este fragmento demuestra cómo se calculan las posiciones. Si el cuerpo tiene como padre al Sol, se calcula desde el centro. Si tiene otro padre (ej. Júpiter), se calcula sumando su rotación a la posición actual del padre.

```
// PASO 1: Calcular posiciones jerárquicas
celestialBodies.forEach(body => {
  if (body.id === 'sun') {
    body.x = centerX;
    body.y = centerY;
  } else if (body.parentId === 'sun') {
    // Planeta orbitando al Sol
    body.x = centerX + Math.cos(body.angle) * body.distance * zoom;
    body.y = centerY + Math.sin(body.angle) * body.distance * zoom;
  } else if (body.parentId) {
    // Luna orbitando a un planeta (Composición de matrices)
    const parent = celestialBodies.find(b => b.id === body.parentId);
    if (parent) {
      body.x = parent.x + Math.cos(body.angle) * body.distance * zoom;
      body.y = parent.y + Math.sin(body.angle) * body.distance * zoom;
    }
  }
  body.screenX = body.x;
  body.screenY = body.y;
});
```

5.2 Detección de Colisión (Clic en el Planeta)

```
canvas.addEventListener('click', (event) => {
  const rect = canvas.getBoundingClientRect();
  const mouseClickX = event.clientX - rect.left;
  const mouseClickY = event.clientY - rect.top;
  for (let body of celestialBodies) {
    // Math.hypot calcula la hipotenusa (distancia entre 2 puntos)
    const dist = Math.hypot(mouseClickX - body.screenX, mouseClickY - body.screenY);
    // Si el clic está dentro del radio del astro (+ margen de 6px)
    if (dist <= (body.radius * zoom) + 6) {
      selectBody(body); // Abre el panel HTML
      return;
    }
  }
});
```

5.3 Renderizado de los Anillos de Saturno

```
case 'saturn':
  // 1. Dibujar la mitad trasera del anillo (de PI a 2*PI)
  ctx.strokeStyle = '#fde047';
  ctx.lineWidth = Math.max(1.5, 3 * zoom);
  ctx.beginPath();
  ctx.ellipse(0, 0, r * 1.8, r * 0.6, 0.2, Math.PI, Math.PI * 2);
  ctx.stroke();
  // 2. Dibujar el cuerpo del planeta
  ctx.fillStyle = '#eab308';
  ctx.beginPath();
  ctx.arc(0, 0, r, 0, Math.PI * 2);
```

```
ctx.fill();
// 3. Dibujar la mitad delantera del anillo (de 0 a PI)
ctx.strokeStyle = '#fde047';
ctx.lineWidth = Math.max(1.5, 3 * zoom);
ctx.beginPath();
ctx.ellipse(0, 0, r * 1.8, r * 0.6, 0.2, 0, Math.PI);
ctx.stroke();
break;
```

6. ESTRUCTURA DEL REPOSITORIO Y COMPILACIÓN

El código está diseñado para ser "Zero-Config" (sin configuración).

Estructura del Repositorio:



Cómo ejecutarlo desde el código:

1. Clona el repositorio: git clone https://github.com/Risamor/Sistema-Solar-2D
2. No requiere compilación (no hay Webpack, Node.js ni preprocesadores).
3. Simplemente abre el archivo src/sistema_solar_2d.html haciendo doble clic o arrastrándolo a tu navegador web.

7. PRUEBAS REALIZADAS Y RESULTADOS DE RENDIMIENTO

Pruebas Funcionales

- **Cálculo anidado:** Se comprobó que la Luna (hija) se mantiene orbitando la Tierra (padre) incluso cuando la Tierra se traslada alrededor del Sol y se aplica Zoom.

- **Interacción:** Se validó que las colisiones (clics) sean precisas en todos los niveles de zoom (0.3x a 5.0x).
- **Inyección de Datos:** Se comprobó que los 7 datos curiosos de cada uno de los 20 astros se renderizan correctamente en el DOM sin desbordar el panel.

Pruebas de Rendimiento (Medido con Chrome DevTools)

- **Frames Per Second (FPS):** Constantes a **60 FPS**.
- **Uso de CPU:** ~12% (en procesador Intel Core i5 de 10.ª generación).
- **Uso de Memoria RAM:** ~45 MB (pestaña activa).
- **Tiempo de carga inicial:** < 0.5 segundos (todo el código está en un solo archivo).

8. LIMITACIONES CONOCIDAS Y MEJORAS POSIBLES

Limitaciones conocidas:

- El código está todo en un solo archivo (sistema_solar_2d.html), lo cual no es la mejor práctica de modularidad para proyectos grandes, aunque fue elegido para facilitar la entrega y ejecución en el Laboratorio 05.
- Las distancias y tamaños de los astros no están a escala real, ya que se prioriza la visibilidad educativa en pantalla (escala didáctica).
- La animación es estrictamente 2D, sin usar transformaciones 3D (Z-buffer).

Mejoras posibles:

- **Modularización:** Separar el código en archivos index.html, styles.css, data.js, math.js y render.js.
- **Modo Matriz Visual:** Añadir un panel en pantalla que muestre explícitamente las matrices de rotación y traslación en tiempo real, para que los estudiantes vean la matemática pura.
- **Audio y Accesibilidad:** Añadir un sintetizador de voz que lea los datos curiosos al hacer clic en un planeta.
- **Optimización:** Usar un Canvas secundario (buffer) para dibujar las estrellas de fondo estáticas y no redibujarlas en cada frame.